Department Scheduling System with AI Integration

Updated Technical Proposal, Data-Gathering Guide, AI Architecture, Dashboard and Analytics Concept

Project direction: A GPT-style conversational scheduling system that generates and manages department schedules while providing dashboards, reports, and data analytics.

1. Project Feasibility

The requested system is technically feasible. A GPT-style interface can be combined with a structured scheduling engine, database, dashboards, reports, and analytics. The language model should assist users rather than independently decide whether a timetable is valid.

A dedicated scheduling/optimization layer should enforce exact constraints such as faculty availability, room capacity, section conflicts, required teaching hours, and department rules. The AI layer can interpret requests, explain results, recommend adjustments, and assist with revisions.

2. Recommended Technology Stack

If the department does not have a required programming language or framework, the following is a suitable starting proposal. The final stack should be confirmed after reviewing school IT standards, deployment requirements, and available infrastructure.

Layer	Recommended Technology	Purpose
Frontend	React + TypeScript	GPT-style interface, spreadsheet-like schedule editor, dashboards, forms, and reports.
Backend	Node.js + TypeScript	Business logic, APIs, authentication, scheduling operations, reporting, and AI orchestration.
Database	PostgreSQL	Faculty, subjects, sections, rooms, schedules, constraints, users, logs, and historical data.
AI Layer	OpenAI API or suitable LLM provider	Natural-language interaction, explanations, recommendations, and controlled schedule assistance.
Scheduling Engine	Constraint-based optimization	Generates feasible schedules while enforcing hard constraints and optimizing defined preferences.
Analytics	React-compatible charting library	Interactive dashboards, workload summaries, room utilization, and trends.
Deployment	Cloud hosting appropriate to requirements	Web access, database hosting, API deployment, monitoring, and backups.

3. GPT-Style User Experience

The preferred interface can resemble a modern AI assistant. Users can type requests in natural language while the system returns structured results such as schedules, tables, charts, reports, and conflict explanations.

- Conversational input: Users provide scheduling instructions or ask questions in natural language.
- Structured output: The system can return schedules, conflict lists, dashboard cards, charts, or reports.
- Schedule actions: Users can request generation, revision, filtering, or conflict explanations.
- Spreadsheet workspace: Users can directly review and edit generated schedules.
- Controlled AI actions: Requested changes are validated by the scheduling engine before being applied.

User	Example
User request	Generate the department schedule using the uploaded faculty availability and room data.
System	Draft schedule generated. Two unresolved conflicts require review.
User request	Show the conflicts and suggest alternatives.
System	Conflict details and feasible alternative slots are presented.

4. Recommended System Architecture

Component	Responsibility
GPT-Style Frontend	Conversation, schedule workspace, dashboards, reports, filters, and user actions.
Application Backend	Authentication, permissions, validation, APIs, workflow management, and audit logging.
AI Orchestration	Converts natural-language requests into structured system actions and explanations.
Scheduling / Optimization Engine	Evaluates assignments, enforces hard constraints, and optimizes defined objectives.
Database	Stores master data, schedules, constraints, user actions, reports, and historical records.
Analytics Layer	Calculates workload, room utilization, conflicts, trends, and approved metrics.

5. AI vs. Scheduling Optimization

AI Layer	Scheduling / Optimization Layer
Understands natural-language requests.	Checks exact scheduling constraints.
Explains conflicts and results.	Prevents faculty, room, and section conflicts.
Suggests possible adjustments.	Tests whether changes are feasible.
Answers questions about schedule data.	Calculates and validates assignments.
Assists the user.	Produces a valid or constraint-compliant schedule.

6. Scheduling Rules and Constraints

Type	Examples
Hard Constraints	No faculty double-booking; no room double-booking; room capacity must accommodate the section; faculty must be available; required subject hours must be scheduled; section conflicts must be prevented.
Soft Constraints	Preferred teaching periods; balanced workload; minimizing gaps; preferred rooms; faculty preferences; minimizing changes from an existing schedule.

7. Data-Gathering Structure

Separate worksheets are recommended instead of one large spreadsheet. This makes validation, database import, and schedule generation easier.

Dataset	Required Information
Faculty	ID, name, program, position, subjects, teaching load, availability, restrictions.
Subjects	Code, name, program, year, units, duration, meetings/week, room requirements.
Sections	Section code, program, year level, student count, enrolled subjects.
Rooms	ID, room number, type, capacity, equipment, availability, restrictions.
Time Slots	Official days, start/end times, breaks, allowed periods.
Faculty Availability	Available/unavailable periods per faculty member.
Subject Requirements	Hours, frequency, preferred periods, room type, special requirements.
Scheduling Rules	Policies, mandatory conditions, restrictions, preferences.
Existing Schedule	Current or previous schedule for reference and comparison.

8. Core Data Templates

Dataset	Suggested Fields
Faculty	Faculty ID; Name; Department/Program; Position; Subjects Handled; Maximum Teaching Load; Preferred Schedule; Unavailable Periods; Special Restrictions.
Subjects	Subject Code; Subject Name; Program; Year Level; Units; Hours per Meeting; Meetings per Week; Preferred Days; Required Room Type; Special Requirements.
Sections	Section Code; Program; Year Level; Number of Students; Subjects; Special Scheduling Rules.
Rooms	Room ID; Room Number; Room Type; Capacity; Equipment; Available Days; Available Time; Special Restrictions.

9. Faculty Availability Template

Faculty ID	Day	Start	End	Status	Notes
FAC-001	Monday	7:30 AM	10:30 AM	Available	
FAC-001	Monday	10:30 AM	12:00 PM	Unavailable	Other assignment
FAC-001	Tuesday	1:00 PM	4:00 PM	Available	

10. Dashboard and Data Analytics

The dashboard can be generated from actual scheduling and department data. Final metrics should be confirmed with the department.

Dashboard / Report	Possible Metrics
Schedule Overview	Classes by day, time, program, year, section, faculty, or room.
Faculty Workload	Teaching hours, assigned subjects, schedule distribution, and remaining load.
Room Utilization	Room usage by day/time, capacity utilization, and unused periods.
Conflict Monitoring	Unresolved conflicts, invalid assignments, and scheduling exceptions.
Subject Distribution	Sections, scheduled hours, and distribution by program/year.
Schedule Changes	Changes, approvals, and revisions.
Historical Analytics	Comparison across semesters if historical schedules are retained.

11. Reports

- Faculty teaching load report
- Room utilization report
- Section/class schedule report
- Subject schedule report
- Conflict and exception report
- Department-wide schedule summary
- Schedule change and revision history
- Exportable PDF/Excel reports, subject to final requirements.

12. Spreadsheet-Like Schedule Workspace

- Grid-based timetable similar to a spreadsheet.
- Filter by faculty, room, section, subject, day, or time.
- Direct editing with validation.
- Visual indicators for conflicts and pending changes.
- AI assistant panel for schedule questions and adjustments.
- Review and confirmation before applying AI-assisted changes.
- Export and print functionality.

13. End-to-End System Flow

Step	Process
1. User Input	User requests schedule generation or asks a scheduling question through the GPT-style interface.
2. Data Validation	System checks whether required faculty, subjects, sections, rooms, availability, and rules are complete.
3. Schedule Generation	Optimization engine generates candidate assignments while enforcing hard constraints.
4. AI Explanation	AI summarizes the schedule and explains conflicts, constraints, or suggested alternatives.
5. User Review	User reviews the schedule in the spreadsheet-like workspace.
6. Adjustment	User requests a change, such as moving a class or avoiding a specific room.
7. Validation	System checks the requested change against applicable constraints.
8. Finalization	Approved schedule is saved and becomes available to dashboards and reports.

14. Data Quality Requirements

- Use consistent IDs for faculty, subjects, sections, and rooms.
- Avoid duplicate records.
- Use one standard time format.
- Distinguish unavailable periods from preferred periods.
- Separate mandatory rules from preferences.
- Document exceptions and special cases.
- Identify official data sources and responsible department personnel.
- Validate student counts, room capacities, faculty loads, and subject requirements.

15. Information the Department Should Provide

After the data-gathering request is approved, the department can provide the following materials for technical analysis:

- Faculty master list
- Subject/course master list
- Section list and student counts
- Room list, room types, capacities, and equipment
- Official time-slot structure
- Faculty availability/unavailability
- Subject scheduling requirements
- Department scheduling policies and restrictions
- Current or previous schedules, if available
- Existing forms, templates, or official scheduling documents
- Any required programming language, framework, database, hosting environment, or school IT standard.

16. Questions to Confirm Before Development

Question	Why It Matters
Is there a required programming language/framework?	Determines technology restrictions and compatibility.
Where will the system be hosted?	Determines deployment, security, database, and infrastructure.
Who are the system users?	Defines roles, permissions, and workflows.
Who can generate, edit, approve, and publish schedules?	Defines authorization and approval controls.
Which rules are mandatory?	Defines hard constraints.
Which preferences should be optimized?	Defines optimization objectives.
What reports and analytics are required?	Defines dashboard and reporting requirements.
Should historical schedules be retained?	Determines historical analytics and retention design.
What information may the AI access?	Defines AI data boundaries and privacy controls.

17. Defining “Optimal Schedule”

An “optimal” schedule must be defined using measurable objectives established by the department. Multiple valid schedules may exist. The system therefore needs a clear distinction between mandatory constraints and preferences.

Possible objectives include minimizing conflicts, reducing schedule gaps, balancing teaching loads, maximizing room utilization, honoring faculty preferences, and minimizing changes from an existing schedule. The department should determine which objectives apply before implementation.

18. Recommended Next Steps

1. Complete and approve the data-gathering request.
2. Collect structured data using the proposed worksheets.
3. Identify hard constraints, soft constraints, user roles, dashboards, and required reports.
4. Confirm technology stack and deployment environment.
5. Design the database, scheduling engine, GPT-style interface, dashboards, and reports.
6. Develop and test using actual department data.
7. Validate generated schedules with department personnel before deployment.

This document is a preliminary technical planning report. The final architecture, AI provider, optimization method, database design, dashboards, reports, and technology stack should be finalized after the department’s actual data, policies, privacy requirements, and infrastructure are reviewed.